

University Of Engineering and Technology, Lahore
Computer Engineering Department

Course Name: Fundamentals of Programming & Data Science	Course Code: CMPE-112L
Assignment Type: Lab Report	Dated: July 1, 2026
Semester: 1st Semester	Session: Spring 2026
Lab/Project/Assignment #: 7	CLOs to be covered: CLO3
Lab Title: List, Tuple	Teacher Name: Hafiz Muhammad Mubashar
Student Name / Roll No: Hashir Zahid (2026(S)-CYS-40)	

Lab Evaluation:

CLO3	Use appropriate data structures to organize, store, and manipulate data efficiently.					
	Level1	Level2	Level3	Level4	Level5	Level6
(5)						
Total						/5

Rubrics for Current Lab Evaluation

Scale	Marks	Level	Rubric
Excellent	5	L1	Submitted all lab tasks, BONUS task, have good

			understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB No 7

Lab Goals / Objectives:

- Understand the concept of sequences in Python
- Learn to create, access, and manipulate lists
- Learn to create, access, and work with tuples
- Understand the differences between lists and tuples
- Master list methods and operations
- Learn list slicing and indexing techniques
- Apply lists and tuples to solve practical problems

Equipment Required:

- Computer System with Python 3.x installed
- Text Editor or IDE (PyCharm, VS Code, or IDLE)
- Operating System: Windows / Linux / macOS

Theory:

1. What are Sequences?

Sequences are ordered collections of items in Python. Each item in a sequence has an index (position) starting from 0. Python provides several built-in sequence types including lists, tuples, and strings.

2. Lists in Python

A list is a mutable (changeable) ordered sequence of items. Lists are defined using square brackets `[]` and can contain items of different data types. Lists are one of the most versatile and commonly used data structures in Python.

```
my_list = [1, 2, 3, "hello", 3.14]
```

3. Creating Lists

Lists can be created in several ways:

- Using square brackets: `my_list = [1, 2, 3]`
- Using the `list()` constructor: `my_list = list((1, 2, 3))`
- Using list comprehension: `my_list = [x for x in range(5)]`
- Empty list: `my_list = []`

4. Accessing List Elements

List elements are accessed using indexing. Python uses zero-based indexing (first element is at index 0). Negative indexing starts from the end (-1 is the last element).

```
fruits = ["apple", "banana", "cherry"]  
print(fruits[0])    # apple  
print(fruits[-1])   # cherry
```

5. List Slicing

Slicing extracts a portion of a list using the syntax `list[start:end:step]`. The end index is exclusive.

```
numbers = [0, 1, 2, 3, 4, 5]
print(numbers[1:4])    # [1, 2, 3]
print(numbers[:3])     # [0, 1, 2]
print(numbers[::2])    # [0, 2, 4]
```

6. List Methods

Python lists provide many built-in methods:

- `append(item)` - Adds an item to the end of the list
- `insert(index, item)` - Inserts an item at a specific position
- `remove(item)` - Removes the first occurrence of an item
- `pop(index)` - Removes and returns the item at the given index
- `sort()` - Sorts the list in ascending order
- `reverse()` - Reverses the order of the list
- `index(item)` - Returns the index of the first occurrence
- `count(item)` - Returns the number of occurrences
- `extend(iterable)` - Adds all items from an iterable to the list
- `clear()` - Removes all items from the list

7. List Operations

- Concatenation (+): Combines two lists
- Repetition (*): Repeats a list
- Membership (in): Checks if an item exists in the list
- Length (len()): Returns the number of items

8. Tuples in Python

A tuple is an immutable (unchangeable) ordered sequence of items. Tuples are defined using parentheses () or just commas. Once created, tuple elements cannot be modified, added, or removed.

```
my_tuple = (1, 2, 3, "hello")
```

9. Why Use Tuples?

- Tuples are faster than lists due to immutability
- Tuples protect data that should not be modified
- Tuples can be used as dictionary keys (lists cannot)
- Tuples are useful for representing fixed collections of items

10. Tuple Operations

Tuples support indexing, slicing, concatenation, repetition, and membership testing just like lists. However, tuples do not support methods that modify the tuple (no append, remove, etc.).

11. Unpacking

Both lists and tuples support unpacking, where elements are assigned to individual variables:

```
coordinates = (10, 20)  
x, y = coordinates
```

12. List vs Tuple

Feature	List	Tuple
Mutable	Yes	No
Syntax	[]	()
Methods	Many	Few

| Speed | Slower | Faster |

| Use Case | Dynamic data | Fixed data |

Lab Work:

Task 1: Append to a List

Write a program that creates a list of fruits and appends a new item to it.

Code Summary:

This task demonstrates list mutability using the `append()` method, which adds a new element to the end of an existing list.

Key Code Lines:

```
# Task 1: Append to a List
fruits = ["Apple", "Banana"]
fruits.append("Orange")
print(fruits)
```

Output:



```
Run  main ×  III ×
C:\Users\itsha\AppData\Local\Python\bin\python.exe "C:\Users\itsha\Desktop\All weeks\name\111.py"
['Apple', 'Banana', 'Orange']
Process finished with exit code 0
```

Task 2: Count List Items with a Function

Write a function that returns the number of items in a list, and test it with a list of fruits.

Code Summary:

This task defines a function that wraps Python's built-in `len()` to count the elements of any list passed to it.

Key Code Lines:

```
# Task 2: Count List Items
def count_items(items):
    return len(items)

fruits = ["Apple", "Banana", "Mango"]
print("Total Items =", count_items(fruits))
```

Output:

A screenshot of a Python IDE terminal window. The window has a title bar with 'Run', 'main', and a Python icon. The terminal shows the command prompt 'C:\Users\itsha\AppData\Local\Python\bin\python.exe "C:\Users\itsha\Desktop\All weeks\name\lll.py"' followed by the output 'Total Items = 3'. Below the output, it says 'Process finished with exit code 0'. The terminal has a dark background and a light-colored text. On the left side of the terminal, there is a vertical toolbar with icons for running, stopping, and other actions.

Task 3: Search for a Name in a List

Write a program that checks whether a student name entered by the user exists in a list of students.

Code Summary:

This task uses the `in` operator to search a list for a matching value and reports whether the student is present or absent.

Key Code Lines:

```
# Task 3: Search for a Name in a List
students = ["Ali", "Hamza", "Ahmed", "Usman"]
name = input("Enter student name: ")

if name in students:
    print("Present")
else:
    print("Absent")
```

Output:

A screenshot of a Python IDE terminal window. The window has a title bar with 'Run', 'main', and a Python icon. The terminal shows the command prompt 'C:\Users\itsha\AppData\Local\Python\bin\python.exe "C:\Users\itsha\Desktop\All weeks\name\lll.py"' followed by the input 'Enter student name: Ali' and the output 'Present'. Below the output, it says 'Process finished with exit code 0'. The terminal has a dark background and a light-colored text. On the left side of the terminal, there is a vertical toolbar with icons for running, stopping, and other actions.

Task 4: Access Tuple Elements

Write a program that stores employee details in a tuple and accesses each field by index.

Code Summary:

This task demonstrates tuple indexing, showing that a tuple's fixed, ordered elements can be retrieved using their position.

Key Code Lines:

```
# Task 4: Access Tuple Elements
employee = ("E101", "Hamza", "Manage", 50000)
print("Employee ID:", employee[0])
print("Name:", employee[1])
print("Job:", employee[2])
print("Salary:", employee[3])
```

Output:



```
Run  main ×  ×
C:\Users\itsha\AppData\Local\Python\bin\python.exe "C:\Users\itsha\Desktop\All weeks\name\l11.py"
Employee ID: E101
Name: Hamza
Job: Manage
Salary: 50000
Process finished with exit code 0
```

Task 5: Sum of List Values

Write a program that stores item prices in a list and calculates the total bill.

Code Summary:

This task uses the built-in `sum()` function to add up all the numeric values stored in a list.

Key Code Lines:

```
# Task 5: Sum of List Values
prices = [120, 80, 250, 60]
print("Items Prices:", prices)
print("Total Bill =", sum(prices))
```

Output:



```
Run  main x  III x
C:\Users\itsha\AppData\Local\Python\bin\python.exe "C:\Users\itsha\Desktop\All weeks\name\l11.py"
Items Prices: [120, 80, 250, 60]
Total Bill = 510
Process finished with exit code 0
```

Task 6: Check Book Availability

Write a program that checks whether a book entered by the user is available in a list of books.

Code Summary:

This task again uses the `in` operator, this time over a list of book titles, to determine availability.

Key Code Lines:

```
# Task 6: Check Book Availability
books = ["Python", "Java", "C++", "Database"]
book = input("Enter book name: ")

if book in books:
    print("Book Available")
else:
    print("Book Not Available")
```

Output:



```
Run  main x  III x
C:\Users\itsha\AppData\Local\Python\bin\python.exe "C:\Users\itsha\Desktop\All weeks\name\l11.py"
Enter book name: Java
Book Available
Process finished with exit code 0
```

Conclusions:

- Successfully understood and created lists and tuples in Python
- Mastered list indexing, slicing, and various list methods

- Learned list comprehension for concise list creation
- Understood the immutability of tuples and their advantages
- Applied tuple unpacking for elegant variable assignment
- Worked with nested lists for multi-dimensional data
- Understood when to use lists (mutable data) vs tuples (immutable data)